



Solution Design Document

PayPal Checkout + BCDC

PayPal Web SDK v6 — Arquitectura Server-Side

PayPal · PayLater · PayPal Credit · BCDC (Guest Card) · STC

Tipo de documento	Solution Design Document (SDD)
Solución	Procesamiento de pagos con PayPal Checkout, PayLater, PayPal Credit y BCDC usando el PayPal Web SDK v6 con arquitectura server-side (BFF)
Componentes técnicos	PayPal Web SDK v6 (web-sdk/v6/core) · paypal-payments · paypal-guest-payments · One-Time Payment Sessions · STC · BFF
Capacidades funcionales	Pago con cuenta PayPal · PayLater · PayPal Credit · BCDC (Guest Card) · STC server-side · Idempotencia por operación (createRequestId / captureRequestId)
APIs REST involucradas	/v1/oauth2/token · /v2/checkout/orders · /v2/checkout/orders/{id} · /v2/checkout/orders/{id}/capture · /v1/risk/transaction-contexts/{merchant_id}/{cmid}
Convenciones	REST → snake_case · SDK JavaScript v6 → camelCase
Audiencia	Solution Architecture · Integration Engineering · Product · e-commerce · Riesgo · Cumplimiento del comercio

Tabla de Contenidos

Tabla de Contenidos	2
1. Resumen Ejecutivo.....	5
2. Contexto de Negocio y Drivers.....	6
2.1 Drivers de negocio	6
2.2 Drivers regulatorios y de cumplimiento	6
3. Alcance de la Solución	8
3.1 Capacidades en alcance.....	8
3.2 Capacidades fuera de alcance.....	8
4. Stakeholders y Audiencia.....	10
5. Glosario de Términos y Acrónimos.....	11
6. Visión General de la Solución.....	13
6.1 Componentes de la solución.....	13
6.2 Identificador de correlación: el CMID	14
6.3 Métodos de pago bajo un único SDK.....	14
7. Requisitos Funcionales.....	16
8. Requisitos No Funcionales	17
9. Arquitectura de la Solución	19
9.1 Diagrama lógico	19
9.2 Reglas no negociables de la arquitectura	19
9.3 Frontera de seguridad por asset	19
10. Prerrequisitos y Configuración del Entorno	21
10.1 Habilitaciones comerciales requeridas	21
10.2 Credenciales y configuración del backend	21
10.3 Requisitos del entorno	22
10.4 Endpoint de configuración pública	22
11. Autenticación OAuth2 (client_credentials).....	23
11.1 Request.....	23
11.2 Response (campos relevantes)	23
11.3 Caché del access_token	23
11.4 Implementación de referencia (Node.js)	24
12. Generación del Client Metadata ID (CMID).....	25
12.1 Reglas de ciclo de vida	25
12.2 Implementación de referencia.....	25

12.3 Validación.....	25
13. Set Transaction Context (STC).....	27
13.1 Endpoint.....	27
13.2 Body — set genérico para Retail	27
13.3 Referencia de campos.....	27
13.4 Manejo de respuesta — comportamiento no bloqueante	28
13.5 Implementación de referencia — STC server-side dentro de Create Order	28
14. Carga del PayPal Web SDK v6	30
14.1 URLs del SDK.....	30
14.2 Inserción en el HTML	30
14.3 Detección de carga	30
15. Inicialización del SDK y Descubrimiento de Métodos Elegibles	31
15.1 paypal.createInstance(...).....	31
15.2 sdkInstance.findEligibleMethods(...)	31
15.3 Detalles específicos de PayLater	32
16. Estructura HTML y Patrón de Activación de Sesión	33
16.1 Botones del comercio	33
16.2 Patrón de activación de sesión — preservación del transient activation	33
16.2.1 Patrón CORRECTO	33
16.2.2 Patrón INCORRECTO (no implementar).....	33
16.3 Patrón BCDC (click handler sobre custom element).....	33
16.4 Forma de la promesa de Create Order	34
16.5 Callbacks de la sesión	34
17–20. Sesiones de Pago.....	36
17. Sesión de PayPal Checkout.....	36
18. Sesión de PayLater	36
19. Sesión de PayPal Credit.....	36
20. Sesión de BCDC — Guest Card Checkout.....	37
20.1 Diferencias clave frente a las otras sesiones.....	37
21. Creación y Captura de Órdenes	39
21.1 Headers HTTP obligatorios.....	39
21.2 Create Order — payload base.....	39
21.3 Reglas de validación del breakdown.....	40
21.4 Capture Order	41

21.5 Contrato frontend → backend	41
21.6 Backend — POST /api/orders	41
22. Orquestación End-to-End	43
22.1 Momento 1 — Inicialización del checkout (una sola vez por sesión)	43
22.2 Momento 2 — Clic del comprador (sin await antes de start)	43
22.3 Momento 3 — onApprove y captura	43
22.4 Regla mnemotécnica	44
23. Puntos de Integración (Mapa de Endpoints)	45
24. Consideraciones de Seguridad	46
24.1 Custodia de credenciales (NFR-02)	46
24.2 Aislamiento PCI (NFR-01)	46
24.3 CSP mínima recomendada (NFR-04)	46
24.4 Logging y manejo de datos sensibles (NFR-08)	46
25. Consideraciones Operativas	48
25.1 Métricas y observabilidad	48
25.2 Idempotencia (NFR-06)	48
25.3 Tolerancia a fallos	49
25.4 Promoción Sandbox → Live	49
26. Estrategia de Pruebas y Entorno Sandbox	51
26.1 Casos de prueba mínimos (TC-01 a TC-16)	51
27. Convenciones de Nombres REST vs SDK	53
28. Asunciones, Dependencias y Restricciones	54
28.1 Asunciones	54
28.2 Dependencias externas	54
28.3 Restricciones	54
29. Riesgos y Mitigaciones	56
30. Criterios de Aceptación y Checklist Pre-producción	58
Apéndice A — Diagnóstico de Errores Frecuentes	60
Apéndice B — Limitaciones y Trabajo Futuro	62
B.1 Limitaciones conocidas	62
B.2 Trabajo futuro sugerido	62

Parte I — Contexto de la solución

1. Resumen Ejecutivo

Este Solution Design Document define la integración de PayPal Checkout + Branded Card-Direct Checkout (BCDC) sobre el PayPal JavaScript SDK v6 (web-sdk/v6/core) en una experiencia de checkout web con arquitectura server-side. La solución habilita cuatro métodos de pago bajo un único SDK y un único patrón de activación de sesión:

- PayPal — pago con cuenta PayPal del comprador.
- PayLater — pago a plazos financiado por PayPal en mercados elegibles.
- PayPal Credit — línea de crédito PayPal en mercados elegibles.
- BCDC — pago con tarjeta de crédito o débito como invitado (sin cuenta PayPal), con la UI de captura hospedada por PayPal.

La arquitectura sigue el patrón Backend-for-Frontend (BFF): el navegador del comprador interactúa con el SDK de PayPal y con un backend privado del comercio, y este último es el único componente autorizado para hablar con la API REST de PayPal. El `client_secret` y el `merchant_id` permanecen en el backend; el frontend recibe únicamente el `client_id` público.

La solución está dirigida a comercios que requieren:

- Cobertura amplia de métodos de pago con un SDK único y una superficie de mantenimiento mínima.
- Aislamiento de credenciales sensibles en el backend, eliminando el `client_secret` del navegador.
- Conversión optimizada mediante el patrón de activación basado en `transient activation`, que evita los bloqueos de pop-up de los navegadores modernos.
- Trazabilidad y correlación de riesgo mediante un Client Metadata ID (CMID) consistente entre frontend, backend, STC y los headers de Create/Capture Order.
- Idempotencia transaccional mediante llaves PayPal-Request-Id independientes por operación (`createRequestId`, `captureRequestId`) para reintentos seguros frente a fallas de red.

El presente documento describe la solución de extremo a extremo: contexto de negocio, requisitos, arquitectura, diseño detallado por componente, orquestación, seguridad, operación, pruebas, riesgos y criterios de aceptación.

2. Contexto de Negocio y Drivers

2.1 Drivers de negocio

Driver	Descripción
Cobertura de métodos de pago con un SDK único	El SDK v6 expone PayPal, PayLater, PayPal Credit y BCDC bajo una única instancia (createInstance) y una API homogénea de sesiones. Esto reduce el costo de mantenimiento frente a integrar cada método por separado.
Conversión y experiencia de usuario	El patrón de activación basado en transient activation preserva el gesto de usuario que dispara la sesión, permitiendo que PayPal abra su pop-up o full-page modal sin que el navegador lo bloquee.
Reducción de fricción para el comprador invitado	BCDC permite cobrar con tarjeta de crédito/débito sin obligar al comprador a crear una cuenta PayPal. La UI de captura es hospedada por PayPal: el comercio solo renderiza un botón.
Aislamiento de credenciales	La arquitectura server-side mueve el client_secret y el merchant_id fuera del navegador, eliminando la categoría de incidente 'secret leaked in HTML' propia de integraciones legacy.
Métodos de financiamiento sin código adicional	PayLater y PayPal Credit se exponen mediante APIs simétricas a la de PayPal Checkout. La elegibilidad por mercado se descubre vía findEligibleMethods, sin necesidad de configuración manual por el comercio.

2.2 Drivers regulatorios y de cumplimiento

Driver	Descripción
PCI DSS	En todos los flujos cubiertos por este SDD, el comercio no procesa, transmite ni almacena datos de cuenta de tarjeta del comprador: PayPal/PayLater/Credit no involucran tarjeta desde la perspectiva del comercio, y en BCDC la captura ocurre íntegramente en la UI hospedada por PayPal. Esto permite mantener el alcance bajo SAQ A.
Custodia de secretos	El client_secret reside únicamente en el backend, gestionado por un secrets manager. Esto facilita el cumplimiento de políticas internas de manejo de credenciales y reduce el alcance de auditorías.
Regulaciones locales para PayLater / Credit	La presentación de financiamiento al comprador está sujeta a regulaciones locales. La UI del flujo PayLater/Credit es

Driver	Descripción
	hospedada por PayPal y cumple los requisitos del mercado en el que el SDK detecta elegibilidad.

3. Alcance de la Solución

3.1 Capacidades en alcance

Capacidad	Descripción
Pago con cuenta PayPal (One-Time)	Cobro con cuenta PayPal del comprador mediante <code>createPayPalOneTimePaymentSession</code> .
Pago con PayLater	Financiamiento PayLater mediante <code>createPayLaterOneTimePaymentSession</code> . La elegibilidad por mercado y producto se descubre con <code>findEligibleMethods</code> y <code>paymentMethods.getDetails('paylater')</code> .
Pago con PayPal Credit	Línea de crédito PayPal mediante <code>createPayPalCreditOneTimePaymentSession</code> en mercados elegibles.
BCDC — Guest Card Checkout	Cobro con tarjeta de crédito/débito como invitado mediante <code>createPayPalGuestOneTimePaymentSession</code> . La UI de captura es hospedada por PayPal; el comercio monta los custom elements <code><paypal-basic-card-container></code> + <code><paypal-basic-card-button></code> y adjunta el listener de clic al botón.
Patrón de activación con transient activation	Construcción de la promesa de Create Order sin await dentro del handler del clic, para preservar el gesto de usuario y evitar bloqueos de pop-up.
Set Transaction Context (STC)	Envío de contexto del comprador al motor de riesgo de PayPal antes de cada Create Order. Operación no bloqueante.
Idempotencia transaccional	Reintentos seguros mediante llaves PayPal-Request-Id independientes por operación (<code>createRequestId</code> para Create, <code>captureRequestId</code> para Capture), persistidas server-side.
Captura post-aprobación	Captura del pago en <code>onApprove</code> mediante POST <code>/v2/checkout/orders/{id}/capture</code> .
Consulta enriquecida post-capture	GET <code>/v2/checkout/orders/{id}</code> para obtener <code>payment_source</code> enriquecido, identificadores de captura y datos para reconciliación.

3.2 Capacidades fuera de alcance

Categoría	Detalle
Capacidades de tarjeta avanzadas	Captura en iframes propios del comercio, control explícito de autenticación reforzada del comprador y su resultado, pagos a

Categoría	Detalle
	meses y reuso de tarjeta entre sesiones quedan fuera del alcance.
Telemetría adicional de riesgo	El SDK v6 inyecta automáticamente la telemetría necesaria; el comercio no integra scripts adicionales de Fraudnet.
Operaciones de back-office	Webhooks, reembolsos, settlement, disputas y conciliación financiera quedan fuera. Requieren un SDD complementario.
Otros métodos de pago	APMs locales y suscripciones recurrentes pueden integrarse con el mismo SDK v6 en soluciones complementarias.
Modos de captura distintos a CAPTURE	AUTHORIZE con captura diferida queda fuera.

4. Stakeholders y Audiencia

Rol	Responsabilidad respecto a este SDD
Solution Architect	Valida la arquitectura, asegura consistencia con el portafolio del comercio y aprueba el documento.
Product Manager (e-commerce)	Valida que el alcance funcional cubre los requisitos de negocio: cobertura de métodos de pago, mercados objetivo y experiencia de checkout.
Integration Engineer (PayPal)	Apoya al comercio en la implementación, valida configuraciones comerciales (habilitación de PayLater/Credit, BCDC, STC) y asiste en la migración desde integraciones legacy.
Integration Engineer / Tech Lead (comercio)	Lidera la implementación técnica y es propietario del código del frontend y del backend.
Equipo de desarrollo del comercio	Implementa y mantiene la integración.
Equipo de Riesgo y Antifraude (comercio)	Define el set de campos additional_data para STC, monitorea métricas de fraude y valida la propagación correcta del CMID.
Equipo de Cumplimiento PCI (comercio)	Verifica que la integración mantiene el alcance SAQ A y que no hay rutas por las que datos de cuenta de tarjeta del comprador alcancen la infraestructura del comercio.
Equipo de QA (comercio)	Ejecuta el plan de pruebas en Sandbox antes de la promoción a Live.
Operaciones (comercio)	Monitorea el checkout en producción, gestiona alertas e incidentes.

5. Glosario de Términos y Acrónimos

Término	Definición
PayPal Web SDK v6	Versión 6 del SDK JavaScript de PayPal, distribuida desde https://www.paypal.com/web-sdk/v6/core (Live) o https://www.sandbox.paypal.com/web-sdk/v6/core (Sandbox). Se carga con <code><script async src="..."></code> sin parámetros de query string.
createInstance	Función del SDK v6 (<code>window.paypal.createInstance({...})</code>) que devuelve una instancia configurada con <code>clientId</code> , <code>components</code> , <code>pageType</code> y <code>locale</code> . Es el primer paso obligatorio antes de cualquier sesión de pago.
findEligibleMethods	Método de la instancia del SDK que devuelve los métodos de pago elegibles para una <code>currencyCode</code> dada. Se usa para decidir qué botones renderizar en la UI.
One-Time Payment Session	Sesión de pago no recurrente. El SDK v6 expone un constructor distinto por método de pago (<code>createPayPalOneTimePaymentSession</code> , <code>createPayLaterOneTimePaymentSession</code> , <code>createPayPalCreditOneTimePaymentSession</code> , <code>createPayPalGuestOneTimePaymentSession</code>). Todas comparten la misma forma: <code>callbacks</code> <code>onApprove</code> , <code>onCancel</code> , <code>onError</code> (y <code>onWarn</code> en BCDC) y un método <code>start(...)</code> .
session.start(options, orderPromise)	Método que activa la sesión y abre la UI hospedada por PayPal. Recibe un objeto <code>options</code> (con <code>presentationMode</code>) y la promesa de <code>Create Order</code> . La promesa no debe estar <code>await</code> -eada en el momento de pasarla. La promesa debe resolver a <code>{ orderId: <ORDER_ID> }</code> .
presentationMode	Atributo de <code>options</code> que controla cómo PayPal presenta su UI. Valores típicos: <code>'auto'</code> , <code>'modal'</code> , <code>'popup'</code> .
<paypal-basic-card-button> / <paypal-basic-card-container>	Custom elements registrados por el SDK v6 cuando el componente <code>'paypal-guest-payments'</code> está habilitado. Son los elementos canónicos para activar la sesión BCDC.
Transient activation	Estado del navegador que indica que el usuario ha realizado un gesto reciente (típicamente un clic). Se consume cuando se abre una ventana o se ejecuta una operación que requiere consentimiento implícito. <code>await</code> antes de invocar <code>session.start</code> consume el estado y los navegadores bloquearán el pop-up.
Order	Recurso REST (<code>/v2/checkout/orders</code>) que representa la intención de cobro: monto, breakdown, items, comprador. Identificado por <code>order_id</code> .
Capture	Operación que ejecuta el cobro real (POST <code>/v2/checkout/orders/{id}/capture</code>).
access_token	Token Bearer OAuth2 que el backend del comercio usa para autenticarse con la API REST de PayPal. Nunca debe enviarse al navegador.

Término	Definición
client_credentials	Grant type de OAuth2 que esta integración usa para obtener el access_token. No se solicita response_type=id_token; el SDK v6 no requiere id_token para inicializarse.
CMID	Client Metadata ID. Identificador alfanumérico de hasta 32 caracteres sin guiones, generado una sola vez por sesión de checkout. Funciona como hilo de correlación entre el frontend, la URL de STC y los headers PayPal-Client-Metadata-Id de Create Order y Capture Order.
STC	Set Transaction Context. Endpoint PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid} mediante el cual el comercio envía contexto del comprador a PayPal Risk antes de cada Create Order. Es no bloqueante.
PayPal-Client-Metadata-Id	Header HTTP que el backend inyecta en Create Order y Capture Order. Contiene el CMID de la sesión.
PayPal-Request-Id	Header HTTP de idempotencia por operación. Cada operación (Create Order, Capture Order) tiene su propia llave (createRequestId, captureRequestId). La misma llave se reutiliza únicamente ante reintentos del mismo request.
BFF	Backend-for-Frontend. Patrón arquitectónico en el que el backend del comercio expone únicamente los endpoints necesarios para el flujo de checkout, custodia los secretos y actúa como proxy autenticado hacia la API REST de PayPal.
BCDC	Branded Card-Direct Checkout. Producto de PayPal para procesar tarjetas de crédito/débito como invitado, con UI de captura hospedada por PayPal. No debe confundirse con ACDC.
pageType	Parámetro de createInstance que indica a PayPal el tipo de página donde el SDK opera ('checkout', 'product-details', 'cart', etc.). Para esta integración el valor es 'checkout'.
locale	Parámetro de createInstance con formato BCP-47 ('es-MX', 'en-US', 'pt-BR', etc.). Define el idioma de la UI hospedada por PayPal.
currencyCode	Parámetro de findEligibleMethods en formato ISO 4217. Determina la elegibilidad de métodos por mercado.
NFR	Non-Functional Requirement. Requisito no funcional.

Parte II — Definición de la solución

6. Visión General de la Solución

La solución se compone de tres planos con responsabilidades estrictamente delimitadas y un identificador de correlación (CMID) que atraviesa los tres durante toda la sesión de checkout.

Plano	Responsabilidad	Lo que NO hace
Navegador del comprador	Genera el CMID, carga el SDK v6, llama a <code>createInstance</code> y <code>findEligibleMethods</code> , instancia las sesiones de pago, renderiza los botones, dispara <code>session.start(...)</code> con la promesa de <code>Create Order</code> y reacciona a los callbacks.	Nunca conoce <code>CLIENT_SECRET</code> , <code>access_token</code> ni <code>MERCHANT_ID</code> . Nunca llama directamente a la API REST de PayPal para operaciones de negocio.
Backend del comercio	Custodia las credenciales, obtiene <code>access_token</code> mediante OAuth2 <code>client_credentials</code> , expone endpoints proxy hacia PayPal, inyecta los headers <code>PayPal-Request-Id</code> (con llaves de idempotencia separadas para <code>Create</code> y <code>Capture</code>) y <code>PayPal-Client-Metadata-Id</code> , ejecuta STC server-side antes de cada <code>Create Order</code> y construye el payload desde el estado del carrito autenticado.	Nunca recibe ni manipula datos de cuenta de tarjeta del comprador. Nunca expone <code>access_token</code> , <code>client_secret</code> ni <code>merchant_id</code> al frontend. Nunca acepta payload de orden, monto, items ni breakdown enviados desde el cliente.
API REST de PayPal	Procesa órdenes, capturas y contexto de riesgo (STC).	Es la única fuente de verdad transaccional.

6.1 Componentes de la solución

Componente	Plano	Función
PayPal Web SDK v6	Navegador	Renderiza la UI hospedada por PayPal, gestiona el ciclo de vida de las sesiones de pago y la comunicación con <code>paypal.com</code> .
Botones del comercio	Navegador	Elementos HTML <code><button></code> propios del comercio que disparan <code>session.start(...)</code> . La marca y estilo son del comercio.

Componente	Plano	Función
OAuth2 Service	Backend	Obtiene y cachea access_token mediante client_credentials.
Orders Proxy	Backend	Crea, consulta y captura órdenes. Inyecta headers de idempotencia y correlación.
STC Caller	Backend	Llama a /v1/risk/transaction-contexts server-side dentro del handler de POST /api/orders, antes de Create Order. No expone un endpoint público al frontend.
API REST PayPal	PayPal	Procesamiento real.

6.2 Identificador de correlación: el CMID

El CMID es un identificador alfanumérico de hasta 32 caracteres sin guiones, generado una sola vez por sesión de checkout. Atraviesa los tres planos:

```

Navegador                                Backend del comercio                                API PayPal
1. genera CMID
2. CMID → body._cmid de /api/orders
   (server-side, en orden estricto)
   PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}
(STC, no bloqueante)
   PayPal-Client-Metadata-Id → POST /v2/checkout/orders
3. CMID → body._cmid de /api/orders/:id/capture
   → PayPal-Client-Metadata-Id → POST
   /v2/checkout/orders/{id}/capture

```

⚠ Sin un CMID consistente entre los tres planos, PayPal Risk no puede correlacionar el contexto de STC con la transacción real, lo que degrada la calidad de la evaluación de riesgo.

💡 Fraudnet en JSv6: A diferencia de integraciones legacy, el SDK v6 inyecta automáticamente la telemetría necesaria. El comercio no debe inyectar scripts adicionales de Fraudnet. El CMID se propaga vía headers HTTP del backend, no vía atributos del <script> del SDK.

6.3 Métodos de pago bajo un único SDK

Método	Constructor de sesión	Componente requerido en createInstance
PayPal	sdkInstance.createPayPalOneTimePaymentSession({ onApprove, onCancel, onError })	'paypal-payments'
PayLater	sdkInstance.createPayLaterOneTimePaymentSession({ onApprove, onCancel, onError })	'paypal-payments'

Método	Constructor de sesión	Componente requerido en createInstance
PayPal Credit	<code>sdkInstance.createPayPalCreditOneTimePaymentSession({ onApprove, onCancel, onError })</code>	'paypal-payments'
BCDC (Guest)	<code>sdkInstance.createPayPalGuestOneTimePaymentSession({ onApprove, onCancel, onWarn, onError })</code>	'paypal-guest-payments'

 El array components que se pasa a createInstance debe incluir explícitamente los componentes correspondientes a los métodos que el comercio expondrá. Para una integración completa: ['paypal-payments', 'paypal-guest-payments'].

7. Requisitos Funcionales

ID	Requisito	Prioridad
RF-01	El sistema debe permitir al comprador completar un cobro mediante su cuenta PayPal a través de <code>createPayPalOneTimePaymentSession</code> .	Obligatorio
RF-02	El sistema debe permitir al comprador completar un cobro con tarjeta de crédito o débito como invitado mediante BCDC (<code>createPayPalGuestOneTimePaymentSession</code>), con la captura de los datos de tarjeta delegada íntegramente a la UI hospedada por PayPal.	Obligatorio
RF-03	El sistema debe ofrecer PayLater y PayPal Credit cuando <code>findEligibleMethods</code> los reporte como elegibles para la moneda y mercado de la sesión.	Obligatorio (cuando aplica)
RF-04	El sistema debe crear cada orden con intent: "CAPTURE" y un breakdown matemáticamente consistente ($\text{item_total} + \text{tax_total} + \text{shipping} - \text{discount} = \text{amount.value}$).	Obligatorio
RF-05	El sistema debe ejecutar la captura del pago en <code>onApprove</code> , mediante una llamada a <code>POST /v2/checkout/orders/{id}/capture</code> ejecutada por el backend del comercio.	Obligatorio
RF-06	El sistema debe activar cada sesión de pago preservando el transient activation del navegador: la promesa de <code>Create Order</code> no debe ser <code>await-eada</code> antes de pasarla a <code>session.start(...)</code> .	Obligatorio
RF-07	El sistema debe transmitir a PayPal Risk el contexto del comprador antes de cada <code>Create Order</code> , sin bloquear el checkout en caso de error de STC.	Obligatorio
RF-08	El sistema debe generar un CMID único por sesión de checkout y propagarlo como header <code>PayPal-Client-Metadata-Id</code> en <code>Create Order</code> y <code>Capture Order</code> , y como segmento de URL en la llamada a STC.	Obligatorio
RF-09	El sistema debe permitir reintentos del comprador dentro de una misma sesión de checkout sin regenerar el CMID.	Obligatorio
RF-10	El sistema debe enriquecer la respuesta post-capture mediante <code>GET /v2/checkout/orders/{id}</code> para obtener identificadores de captura y datos para reconciliación.	Obligatorio
RF-11	El sistema debe mostrar al comprador un mensaje claro y accionable cuando un pago falle (<code>onError</code>) o sea cancelado por el comprador (<code>onCancel</code>).	Obligatorio
RF-12	El sistema debe degradar elegantemente cuando un método de pago no resulte elegible: el botón correspondiente no se renderiza y la UI no presenta opciones inválidas al comprador.	Obligatorio

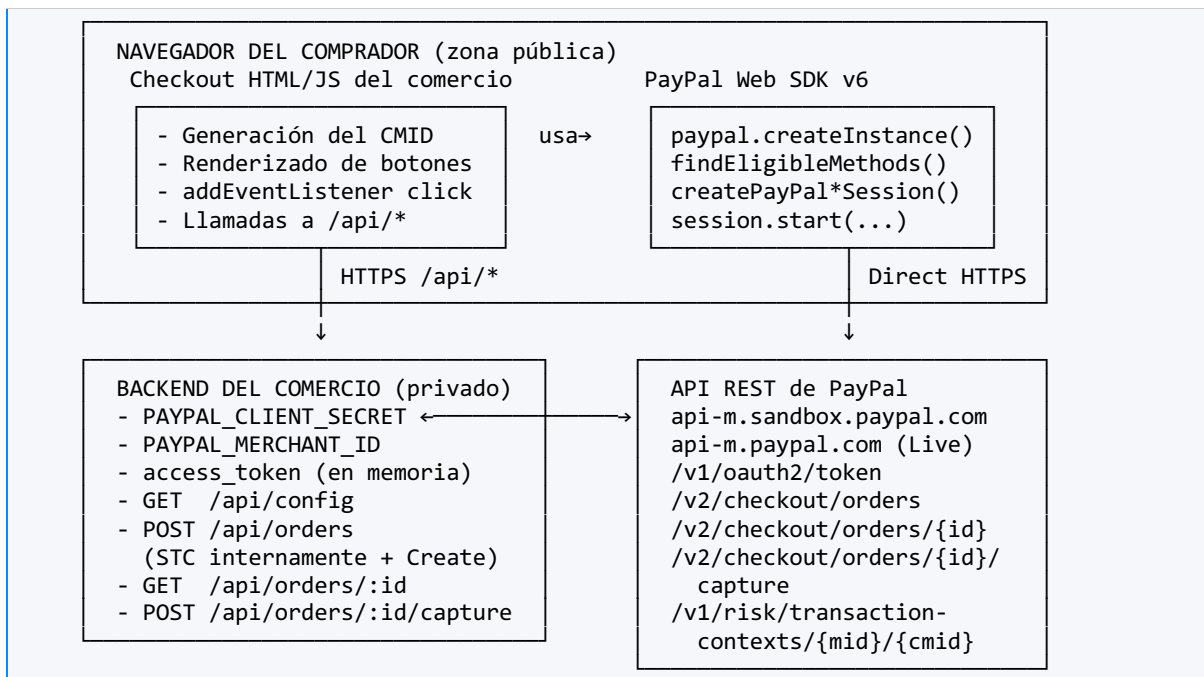
8. Requisitos No Funcionales

ID	Categoría	Requisito
NFR-01	Seguridad — PCI DSS	El comercio no debe procesar, transmitir ni almacenar datos de cuenta de tarjeta del comprador en ningún punto del flujo. La integración debe permitir cumplimiento bajo SAQ A.
NFR-02	Seguridad — secretos	PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID y access_token nunca deben transmitirse al navegador, ni almacenarse en código fuente versionado, ni aparecer en logs.
NFR-03	Seguridad — transporte	Todas las comunicaciones (navegador ↔ backend, backend ↔ PayPal) deben usar TLS 1.2 o superior.
NFR-04	Seguridad — CSP	El sitio debe declarar una Content Security Policy que liste explícitamente los orígenes de PayPal: www.paypal.com, www.sandbox.paypal.com, www.paypalobjects.com, c.paypal.com, api-m.paypal.com.
NFR-05	Disponibilidad	La caída de servicios accesorios (STC) no debe interrumpir el flujo de pago. STC opera de forma no bloqueante.
NFR-06	Idempotencia	El sistema debe garantizar que reintentos de Create Order o Capture Order no generen órdenes ni capturas duplicadas mediante PayPal-Request-Id, usando llaves independientes por operación (createRequestId para Create, captureRequestId para Capture).
NFR-07	Latencia (p95)	El tiempo total entre el clic del botón y la respuesta al usuario no debe exceder los SLOs internos del comercio. La caché del access_token (§11.3) y el procesamiento no bloqueante de STC son esenciales para cumplirlo.
NFR-08	Trazabilidad	El sistema debe loguear order_id, capture_id, status, código de error PayPal y CMID, sin loguear secretos del backend.
NFR-09	Internacionalización	El SDK debe inicializarse con el locale correspondiente al mercado destino y los mensajes de error deben traducirse al idioma del comprador.
NFR-10	Accesibilidad	Los botones del comercio que activan las sesiones deben cumplir WCAG 2.1 AA: foco visible, etiquetas accesibles, contraste suficiente, área táctil mínima 44×44 px.
NFR-11	Escalabilidad	El backend debe cachear el access_token hasta el 90% de expires_in para no saturar /v1/oauth2/token bajo carga.

ID	Categoría	Requisito
NFR-12	Compatibilidad de navegador	La integración debe ejecutar findEligibleMethods antes de renderizar cualquier botón y omitir aquellos métodos que no resulten elegibles.
NFR-13	Auditabilidad	Cada transacción debe ser reconstruible a partir de los logs: CMID, createRequestId, captureRequestId, order_id, invoice_id, custom_id.
NFR-14	Preservación del transient activation	El handler del clic del botón no debe contener await antes de invocar session.start(...). La promesa de Create Order debe construirse e inmediatamente pasarse al SDK.

9. Arquitectura de la Solución

9.1 Diagrama lógico



9.2 Reglas no negociables de la arquitectura

1. PAYPAL_CLIENT_SECRET reside únicamente en variables de entorno del backend. Jamás se versiona, jamás se envía al navegador.
2. El navegador nunca llama directamente a api-m.paypal.com para operaciones de negocio. Toda comunicación REST pasa por el backend del comercio.
3. El navegador nunca recibe el access_token. Recibe únicamente el client_id público (vía GET /api/config) y los identificadores devueltos por las llamadas proxy (order_id, capture_id).
4. Los datos de cuenta de tarjeta del comprador nunca tocan el comercio. En BCDC la captura ocurre íntegramente en la UI hospedada por PayPal.
5. Los datos de carrito, comprador y dirección de envío nunca son hardcoded en el código del frontend. Proviene de la sesión autenticada y del estado del carrito en el backend.
6. El backend no devuelve campos sensibles (client_secret, access_token, merchant_id) al frontend bajo ninguna circunstancia.

9.3 Frontera de seguridad por asset

Asset	Frontend	Backend	API PayPal
CLIENT_ID	Sí	Sí	—
CLIENT_SECRET	No	Sí (env)	—
MERCHANT_ID	No	Sí (env)	—
access_token	No	Sí (memoria)	—
Datos de cuenta de tarjeta	UI hospedada PayPal (BCDC)	No	Sí
order_id	Sí (referencia)	Sí	Sí
capture_id	Sí (referencia)	Sí	Sí
CMID	Sí (genera)	Sí (recibe en body)	Sí (header)

10. Prerrequisitos y Configuración del Entorno

10.1 Habilitaciones comerciales requeridas

Capacidad	Aplica a
PayPal Checkout (One-Time Payments)	Pago con cuenta PayPal.
BCDC (Branded Card-Direct Checkout)	Pago con tarjeta de crédito/débito como invitado.
PayLater	Cuando el comercio desee ofrecer PayLater en mercados elegibles.
PayPal Credit	Cuando el comercio desee ofrecer la línea de crédito PayPal.
Set Transaction Context (STC)	Acceso al endpoint /v1/risk/transaction-contexts.

10.2 Credenciales y configuración del backend

Variable de entorno	Origen	Visibilidad
PAYPAL_CLIENT_ID	PayPal Developer Dashboard → app del comercio.	Pública (puede exponerse al frontend mediante GET /api/config).
PAYPAL_CLIENT_SECRET	PayPal Developer Dashboard → app del comercio.	Secreta — solo backend.
PAYPAL_MERCHANT_ID	Perfil de la cuenta merchant en PayPal.	Privada — solo backend (requerida para STC).
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com (Sandbox) · https://api-m.paypal.com (Live).	Privada.
PAYPAL_SDK_URL	https://www.sandbox.paypal.com/web-sdk/v6/core (Sandbox) · https://www.paypal.com/web-sdk/v6/core (Live).	Pública (puede exponerse al frontend).

 Las credenciales y URLs de Sandbox y Live son distintas. PAYPAL_API_BASE, PAYPAL_SDK_URL, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET y PAYPAL_MERCHANT_ID deben corresponder al mismo entorno. Una mezcla produce 401 Unauthorized en la primera llamada a OAuth o errores opacos en la inicialización del SDK.

10.3 Requisitos del entorno

- TLS / HTTPS obligatorio en producción. El SDK v6 no opera sobre HTTP.
- Content Security Policy (CSP): script-src https://www.paypal.com https://www.paypalobjects.com https://c.paypal.com; frame-src https://www.paypal.com; connect-src https://api-m.paypal.com https://api-m.sandbox.paypal.com (más equivalentes Sandbox cuando aplica).
- Soporte de Web Crypto API en el navegador (para crypto.randomUUID()). Para soportar navegadores legacy, implementar fallback (ver §12.2).
- Soporte de pop-ups del dominio del comercio. El patrón de activación basado en transient activation (§16) está diseñado para que el navegador permita el pop-up sin requerir configuración del usuario.

10.4 Endpoint de configuración pública

```
GET /api/config
→ 200 OK
{
  "clientId": "&lt;PAYPAL_CLIENT_ID&gt;",
  "sdkUrl": "&lt;PAYPAL_SDK_URL&gt;"
}
```

 Este endpoint nunca debe devolver client_secret, access_token ni merchant_id. Tests automatizados deben validar que la respuesta no contenga estas claves.

Parte III — Diseño detallado

11. Autenticación OAuth2 (client_credentials)

El backend del comercio es el único componente autorizado para hablar OAuth2 con PayPal. El propósito de este paso es obtener un access_token Bearer para que el backend invoque la API REST de PayPal.

🔒 A diferencia de integraciones basadas en el SDK clásico, esta solución no requiere id_token. El SDK v6 se inicializa únicamente con el client_id público; no hay atributo data-sdk-client-token. El grant es estrictamente client_credentials.

11.1 Request

```
POST {PAYPAL_API_BASE}/v1/oauth2/token
Authorization: Basic <BASE64(CLIENT_ID:CLIENT_SECRET)>
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
```

11.2 Response (campos relevantes)

```
{
  "access_token": "<ACCESS_TOKEN>",
  "expires_in": 32400,
  "token_type": "Bearer"
}
```

11.3 Caché del access_token

Práctica	Justificación
Cachear el access_token en memoria del proceso del backend hasta el 90% de expires_in y refrescarlo proactivamente.	Evita llamadas innecesarias a /v1/oauth2/token y reduce latencia en el camino crítico de Create Order.
No persistir el access_token en disco, base de datos ni logs.	Token Bearer con poder transaccional; su exposición compromete la cuenta.
Refrescar inmediatamente ante un 401 de cualquier llamada y reintentar una vez.	Cubre el edge case de token revocado por rotación de credenciales.

Práctica	Justificación
Invalidar la caché ante una rotación manual de CLIENT_SECRET.	Una rotación deja el token cacheado inválido; el backend debe forzar refresh en la próxima llamada.

11.4 Implementación de referencia (Node.js)

```
let cachedToken = null;
let cachedTokenExpiresAt = 0;

async function getAccessToken() {
  const now = Date.now();
  if (cachedToken && now < cachedTokenExpiresAt) return cachedToken;

  const credentials = Buffer.from(
    `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`
  ).toString("base64");

  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v1/oauth2/token`, {
    method: "POST",
    headers: { "Authorization": `Basic ${credentials}`,
      "Content-Type": "application/x-www-form-urlencoded" },
    body: "grant_type=client_credentials"
  });

  if (!response.ok) throw new Error(`OAuth failed: ${response.status}`);

  const json = await response.json();
  cachedToken = json.access_token;
  cachedTokenExpiresAt = now + (json.expires_in * 1000 * 0.9);
  return cachedToken;
}
```


12. Generación del Client Metadata ID (CMID)

El CMID es un identificador único alfanumérico de hasta 32 caracteres sin guiones que actúa como hilo de correlación entre: la URL del endpoint de STC, el header PayPal-Client-Metadata-Id enviado al Create Order, y el header PayPal-Client-Metadata-Id enviado al Capture Order.

12.1 Reglas de ciclo de vida

Regla	Detalle
Único por sesión de checkout	Se genera una sola vez al inicializar la página del checkout.
Persistente entre reintentos	Si el comprador falla un pago y reintenta dentro de la misma sesión, el CMID no se regenera.
Persistente entre métodos de pago	Si el comprador alterna entre PayPal, PayLater, Credit y BCDC en la misma sesión, el CMID se conserva.
Se regenera en transacción nueva	Solo después de que la transacción anterior fue completada o cancelada definitivamente se genera un nuevo CMID en una nueva sesión de checkout.
Se propaga server-side por el backend	El frontend lo entrega al backend en el body de Create Order/Capture Order; el backend lo reenvía como header HTTP a PayPal.

12.2 Implementación de referencia

```
function generateCMID() {
  if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function") {
    return crypto.randomUUID().replace(/-/g, ""); // 32 caracteres hex
  }
  return "xxxxxxxxxx4xxxyxxxxxxxxxxxxxxxx".replace(/[xy]/g, (c) => {
    const r = (Math.random() * 16) | 0;
    return (c === "x" ? r : (r & 0x3) | 0x8).toString(16);
  });
}
const cmid = generateCMID();
```

12.3 Validación

```
const isValidCMID = (cmid) => {
  typeof cmid === "string" && cmid.length >= 1 && cmid.length <= 32
  && /^[0-9a-zA-Z]+$/.test(cmid);
}
```

💡 El backend debe aplicar esta validación al `_cmid` recibido del frontend antes de propagarlo como header HTTP a PayPal. Un CMID malformado debe rechazarse con 400 Bad Request.

13. Set Transaction Context (STC)

STC permite al comercio enviar contexto del comprador a PayPal Risk antes de cada Create Order. Se ejecuta server-side dentro del handler de POST /api/orders, secuenciada antes de la llamada a POST /v2/checkout/orders y envuelta en try/catch. La operación es no bloqueante.

13.1 Endpoint

```
PUT {PAYPAL_API_BASE}/v1/risk/transaction-contexts/{MERCHANT_ID}>/{CMID}>
Authorization: Bearer {ACCESS_TOKEN}>
Content-Type: application/json
```

13.2 Body — set genérico para Retail

```
{
  "additional_data": [
    { "key": "sender_account_id", "value": "{ID_DEL_COMPRADOR_EN_LA_PLATAFORMA}" },
    { "key": "sender_first_name", "value": "{NOMBRE_DEL_COMPRADOR}" },
    { "key": "sender_last_name", "value": "{APELLIDO_DEL_COMPRADOR}" },
    { "key": "sender_email", "value": "{EMAIL_DEL_COMPRADOR}" },
    { "key": "sender_phone", "value": "{TELEFONO_SOLO_DIGITOS}" },
    { "key": "sender_country_code", "value": "{PAIS_ISO_ALPHA2}" },
    { "key": "sender_create_date", "value": "{FECHA_DE_ALTA_DEL_USUARIO}" },
    { "key": "highrisk_txn_flag", "value": "0" },
    { "key": "vertical", "value": "{VERTICAL_DEL_NEGOCIO}" }
  ]
}
```

13.3 Referencia de campos

Campo	Tipo	Descripción	Valores aceptados
sender_account_id	string	ID único del comprador en la plataforma del comercio.	Alfanumérico estable entre sesiones.
sender_first_name	string	Nombre registrado del comprador.	Alfanumérico.
sender_last_name	string	Apellido registrado del comprador.	Alfanumérico.
sender_email	string	Email validado del comprador.	Formato RFC 5322.

Campo	Tipo	Descripción	Valores aceptados
sender_phone	string	Teléfono del comprador, solo dígitos, sin formato.	[0-9]+
sender_country_code	string	País del comprador en ISO 3166-1 Alpha-2.	MX, US, BR, etc.
sender_create_date	string	Fecha de alta del usuario en la plataforma del comercio.	yyyy-mm-ddThh:mm:ssZ yyyy-mm-dd yyyymmdd
highrisk_txn_flag	string	Indica si la transacción es de alto riesgo.	"0" = normal, "1" = alto riesgo.
vertical	string	Vertical del negocio.	Retail, Travel, Gaming, etc.

💡 Industry packs: El set anterior es el genérico para Retail. Verticales como Travel, OTAs, Financial Services, Gaming y plataformas reguladas requieren campos adicionales. Solicitar el industry pack al Integration Engineer.

13.4 Manejo de respuesta — comportamiento no bloqueante

Status	Significado	Acción
200	OK. Contexto registrado.	Continuar con Create Order.
400	Body inválido.	Loguear el error con detalle, continuar con Create Order.
401	Sin permisos o token expirado.	Refrescar token, loguear, continuar con Create Order.
5xx	Error interno de PayPal.	Loguear, continuar con Create Order.

🔒 STC nunca puede bloquear el checkout. Una falla de STC reduce la calidad de la evaluación de riesgo pero no impide procesar la transacción.

13.5 Implementación de referencia — STC server-side dentro de Create Order

```
// Función auxiliar callSTC en el backend
async function callSTC({ cmid, additionalData }) {
  const accessToken = await getAccessToken();
  const response = await fetch(
    `${process.env.PAYPAL_API_BASE}/v1/risk/transaction-
    contexts/${process.env.PAYPAL_MERCHANT_ID}/${cmid}`,
    { method: "PUT",
```

```
    headers: { "Authorization": `Bearer ${accessToken}`, "Content-Type":
"application/json" },
    body: JSON.stringify({ additional_data: additionalData }) }
  );
  if (!response.ok) throw new Error(`STC failed: ${response.status}`);
}

// Uso dentro de POST /api/orders
try {
  await callSTC({
    cmid: _cmid,
    additionalData: buildAdditionalDataFromTrustedState(req.user)
  });
} catch (err) {
  logger.warn({ err, cmid: _cmid }, "STC failed; continuing checkout");
}
// Continuar con Create Order independientemente del resultado de STC
```

 STC se ejecuta antes de cada Create Order, sin distinción del método de pago. El frontend no invoca STC directamente; solo envía el `_cmid` al backend en el body de POST `/api/orders`.

14. Carga del PayPal Web SDK v6

El SDK v6 se carga con un `<script async>` directamente en el HTML del checkout. No acepta parámetros de query string como el SDK clásico.

14.1 URLs del SDK

Entorno	URL
Sandbox	https://www.sandbox.paypal.com/web-sdk/v6/core
Live	https://www.paypal.com/web-sdk/v6/core

14.2 Inserción en el HTML

```
<script async src="&lt;PAYPAL_SDK_URL&gt;"&gt;</script>
```

🔒 No agregar atributos `data-*` al `<script>`. El SDK v6 no se configura mediante atributos del tag; toda la configuración pasa por `paypal.createInstance(...)`.

14.3 Detección de carga

```
function loadSDK(sdkUrl) {
  return new Promise((resolve, reject) => {
    const s = document.createElement("script");
    s.async = true;
    s.src = sdkUrl;
    s.onload = () => resolve(window.paypal);
    s.onerror = () => reject(new Error("PayPal SDK failed to load"));
    document.body.appendChild(s);
  });
}
```

⚠️ Un cambio entre Sandbox y Live requiere recargar la página. El SDK v6 registra custom elements y estos no pueden re-registrarse en la misma sesión. Cualquier lógica que pretenda alternar entornos en caliente fallará silenciosamente.

15. Inicialización del SDK y Descubrimiento de Métodos Elegibles

15.1 paypal.createInstance(...)

```
const sdkInstance = await window.paypal.createInstance({
  clientId:      "<PAYPAL_CLIENT_ID>",
  components:    ["paypal-payments", "paypal-guest-payments"],
  pageType:      "checkout",
  locale:        "<LOCALE_BCP47>",
  clientMetadataId: cmid // opcional pero recomendado
});
```

Parámetro	Valor	Notas
clientId	<PAYPAL_CLIENT_ID>	Recibido del backend en GET /api/config. No es un secreto: puede aparecer en HTML, herramientas de desarrollador y logs sin riesgo.
components	['paypal-payments', 'paypal-guest-payments']	paypal-payments habilita PayPal, PayLater y PayPal Credit. paypal-guest-payments habilita BCDC. Para una integración completa, ambos.
pageType	'checkout'	Indica al SDK que opera en página de checkout (afecta métricas y presentación).
locale	'es-MX', 'en-US', 'pt-BR', etc.	Idioma de la UI hospedada por PayPal. Formato BCP-47 con guion. Debe corresponder al mercado del comprador.
clientMetadataId	El mismo CMID generado en §12 (opcional).	Correlaciona la telemetría que el SDK envía a PayPal Risk con la sesión, igual que los headers PayPal-Client-Metadata-Id server-side.

 createInstance devuelve una Promise. Sí debe ser await-eada (esto ocurre en la fase de inicialización, no en el handler del clic; no hay riesgo de consumir el transient activation).

15.2 sdkInstance.findEligibleMethods(...)

```
const paymentMethods = await sdkInstance.findEligibleMethods({ currencyCode:
  "<MONEDA_ISO_4217>" });

if (paymentMethods.isEligible("paypal")) renderPayPalButton();
if (paymentMethods.isEligible("paylater")) renderPayLaterButton();
if (paymentMethods.isEligible("credit")) {
```

```
const creditDetails = paymentMethods.getDetails("credit");
renderPayPalCreditButton(creditDetails);
}
if (paymentMethods.isEligible("guest")) renderGuestCardButton();
```

🔒 No renderizar un botón cuyo método no es elegible. Hacerlo produce errores opacos en `session.start(...)` y daña la experiencia del comprador.

💡 Las claves que acepta `isEligible(...)` y `getDetails(...)` son: "paypal", "paylater", "credit" (no "paypal-credit") y "guest". La etiqueta de marketing "PayPal Credit" se mantiene únicamente en la UI; en código siempre se usa la clave "credit".

15.3 Detalles específicos de PayLater

```
const paylaterDetails = paymentMethods.getDetails("paylater");
// paylaterDetails: { productCode: "<...>", countryCode: "<...>" }
```


16. Estructura HTML y Patrón de Activación de Sesión

16.1 Botones del comercio

```
<!-- PayPal, PayLater, PayPal Credit: botones del comercio -->
<button id="btn-paypal" type="button">Pagar con PayPal</button>
<button id="btn-paylater" type="button">Pagar con PayLater</button>
<button id="btn-paypal-credit" type="button">Pagar con PayPal Credit</button>

<!-- BCDC: custom element hospedado por PayPal -->
<paypal-basic-card-container>
  <paypal-basic-card-button id="paypal-basic-card-button"></paypal-basic-card-button>
</paypal-basic-card-container>

<div id="payment-result" role="status" aria-live="polite"></div>
```

🔒 Para que `<paypal-basic-card-container>` y `<paypal-basic-card-button>` se registren correctamente, el componente 'paypal-guest-payments' debe estar incluido en el array `components` de `createInstance(...)` y el SDK debe haber terminado de cargar antes de que el HTML se renderice.

16.2 Patrón de activación de sesión — preservación del transient activation

🚨 Esta es la regla más crítica de la integración. Cualquier `await` antes de invocar `session.start(...)` consume el estado y el navegador bloquea la apertura del pop-up.

16.2.1 Patrón CORRECTO

```
document.getElementById("btn-paypal").addEventListener("click", () => {
  // Construir la promesa SIN await — preserva el transient activation
  const orderPromise = createOrder({ paymentMethod: "paypal" });
  session.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});
```

16.2.2 Patrón INCORRECTO (no implementar)

```
// INCORRECTO — el await consume el transient activation
document.getElementById("btn-paypal").addEventListener("click", async () => {
  const orderId = await createOrder({ paymentMethod: "paypal" }); // NUNCA HACER ESTO
  session.start({ presentationMode: "auto" }, Promise.resolve({ id: orderId }))
    .catch(onError);
});
```

16.3 Patrón BCDC (click handler sobre custom element)

```
const cardBtn = document.getElementById("paypal-basic-card-button");
cardBtn.addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "guest" });
  guestSession.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});
```

💡 El SDK v6 admite variantes de apertura programática con `targetElement`. Esas variantes no son el flujo canónico de este SDD; cualquier desviación debe validarse con el Integration Engineer.

16.4 Forma de la promesa de Create Order

```
async function createOrder({ paymentMethod }) {
  const response = await fetch("/api/orders", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      _cmid: cmid,
      paymentMethod: paymentMethod, // 'paypal' | 'paylater' | 'credit' | 'guest'
      cartId: getCurrentCartId()
    })
  });
  if (!response.ok) throw new Error(`Create Order failed: ${response.status}`);
  const order = await response.json();
  return { orderId: order.id }; // El SDK v6 exige la forma { orderId: <ORDER_ID> }
}
```

🔒 La promesa debe resolver a `{ orderId: <ORDER_ID> }`. No usar `{ id: ... }`; el SDK v6 espera específicamente la propiedad `orderId`. Devolver la forma incorrecta produce que `data.orderId` llegue `undefined` al callback `onApprove`.

16.5 Callbacks de la sesión

Callback	PayPal / PayLater / Credit	BCDC	Función
onApprove(data)	Sí	Sí	El comprador aprobó el pago. <code>data.orderId</code> contiene el <code>order_id</code> . El comercio dispara la captura.
onCancel(data)	Sí	Sí	El comprador cerró la UI sin completar el pago. Mostrar mensaje sobrio y permitir reintento.
onError(err)	Sí	Sí	Error técnico durante la sesión. Loguear y mostrar mensaje accionable.

Callback	PayPal / PayLater / Credit	BCDC	Función
onWarn(warn)	—	Sí	Advertencia no fatal específica de BCDC. El SDK ya muestra feedback al usuario; el comercio típicamente solo loguea.

 En BCDC, no omitir onWarn. El callback existe específicamente para advertencias de la UI hospedada; ignorarlo deja al comercio sin telemetría sobre fricción de captura.

17–20. Sesiones de Pago

Las sesiones de PayPal, PayLater, PayPal Credit y BCDC comparten el mismo patrón: inicialización antes del clic, activación en el handler del clic con la promesa de Create Order, y captura en onApprove.

17. Sesión de PayPal Checkout

```
const paypalSession = sdkInstance.createPayPalOneTimePaymentSession({
  onApprove: async (data) => {
    const result = await captureOrder(data.orderId);
    showSuccess(result);
  },
  onCancel: (data) => { showInfo("Pago cancelado por el comprador."); },
  onError: (err) => { console.error(err); showError("No fue posible completar el pago. Intente nuevamente."); }
});

document.getElementById("btn-paypal").addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "paypal" });
  paypalSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
});
```

18. Sesión de PayLater

```
if (!paymentMethods.isEligible("paylater")) {
  document.getElementById("btn-paylater").style.display = "none";
} else {
  const paylaterSession = sdkInstance.createPayLaterOneTimePaymentSession({
    onApprove: async (data) => {
      const result = await captureOrder(data.orderId);
      showSuccess(result);
    },
    onCancel: (data) => { showInfo("Financiamiento cancelado."); },
    onError: (err) => { console.error(err); showError("Error al iniciar PayLater."); }
  });
  document.getElementById("btn-paylater").addEventListener("click", () => {
    const orderPromise = createOrder({ paymentMethod: "paylater" });
    paylaterSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
  });
}
```

19. Sesión de PayPal Credit

```
if (!paymentMethods.isEligible("credit")) {
  document.getElementById("btn-paypal-credit").style.display = "none";
} else {
```

```
const creditSession = sdkInstance.createPayPalCreditOneTimePaymentSession({
  onApprove: async (data) => {
    const result = await captureOrder(data.orderId);
    showSuccess(result);
  },
  onCancel: (data) => showInfo("Operación cancelada."),
  onError: (err) => { console.error(err); showError("Error al iniciar PayPal Credit."); }
});
document.getElementById("btn-paypal-credit").addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "credit" });
  creditSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
});
```

20. Sesión de BCDC — Guest Card Checkout

20.1 Diferencias clave frente a las otras sesiones

Aspecto	PayPal / PayLater / Credit	BCDC
Componente en createInstance	'paypal-payments'	'paypal-guest-payments'
Constructor	createPayPal*OneTimePaymentSession	createPayPalGuestOneTimePaymentSession
Callbacks	onApprove, onCancel, onError	onApprove, onCancel, onWarn, onError
Elemento que dispara la sesión	<button> HTML del comercio	<paypal-basic-card-button> (custom element registrado por el SDK)
Captura de tarjeta	No aplica	UI hospedada por PayPal

```
if (!paymentMethods.isEligible("guest")) {
  document.querySelector("paypal-basic-card-container").style.display = "none";
} else {
  const guestSession = sdkInstance.createPayPalGuestOneTimePaymentSession({
    onApprove: async (data) => {
      const result = await captureOrder(data.orderId);
      showSuccess(result);
    },
    onCancel: (data) => showInfo("Pago con tarjeta cancelado."),
    onWarn: (warn) => console.warn("BCDC warning:", warn),
    onError: (err) => { console.error(err); showError("No fue posible procesar la tarjeta."); }
  });
  document.getElementById("paypal-basic-card-button").addEventListener("click", () => {
    const orderPromise = createOrder({ paymentMethod: "guest" });
    guestSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
  });
}
```

```
}
```

💡 En el flujo BCDC del SDK v6 cubierto por este SDD, el comercio no recibe ni evalúa `liabilityShift` ni `enrollment_status`. La eventual ejecución de 3DS por parte de PayPal es transparente.

21. Creación y Captura de Órdenes

21.1 Headers HTTP obligatorios

Header	Create	Capture	Descripción
Authorization: Bearer <ACCESS_TOKEN>	Sí	Sí	Token Bearer obtenido en §11.
Content-Type: application/json	Sí	Sí	El body es JSON.
PayPal-Request-Id	<CREATE_REQUEST_ID>	<CAPTURE_REQUEST_ID>	Idempotencia por operación. Llave distinta para Create y Capture. La misma llave se reutiliza solo ante reintentos del mismo request.
PayPal-Client- Metadata-Id: <CMID>	Sí	Sí	Vincula la transacción con el contexto de STC. Mismo CMID en Create y Capture (identificador de la sesión, no de la operación).

 Las llaves <CREATE_REQUEST_ID> y <CAPTURE_REQUEST_ID> son independientes. Reutilizar la misma llave para Create y Capture viola la guía de idempotencia de PayPal y puede producir comportamiento inconsistente.

21.2 Create Order — payload base

```
{
  "intent": "CAPTURE",
  "application_context": {
    "brand_name": "&lt;NOMBRE_VISIBLE_DEL_COMERCIO&gt;",
    "locale": "&lt;LOCALE_BCP47&gt;",
    "shipping_preference": "SET_PROVIDED_ADDRESS",
    "user_action": "PAY_NOW",
    "return_url": "&lt;URL_DE_RETORNO_REAL_DEL_COMERCIO&gt;",
```

```

    "cancel_url": "<URL_DE_CANCELACION_REAL_DEL_COMERCIO>",
  },
  "payer": {
    "email_address": "<EMAIL_DEL_COMPRAADOR>",
    "name": { "given_name": "<NOMBRE_DEL_COMPRAADOR>", "surname":
"<APELLIDO_DEL_COMPRAADOR>" },
    "phone": { "phone_type": "MOBILE", "phone_number": { "national_number":
"<TELEFONO_SOLO_DIGITOS>" } }
  },
  "purchase_units": [{
    "invoice_id": "<INVOICE_ID_UNICO_DEL_COMERCIO>",
    "custom_id": "<ORDER_ID_INTERNO_DEL_COMERCIO>",
    "description": "<DESCRIPCION_BREVE_DEL_PEDIDO>",
    "amount": {
      "currency_code": "<MONEDA_ISO_4217>",
      "value": "<TOTAL_DEL_CARRITO>",
      "breakdown": {
        "item_total": { "currency_code": "<MONEDA>", "value":
"<SUMA_UNIT_AMOUNT_X_QTY>" },
        "tax_total": { "currency_code": "<MONEDA>", "value":
"<SUMA_TAX_X_QTY>" },
        "shipping": { "currency_code": "<MONEDA>", "value":
"<COSTO_DE_ENVIO>" },
        "discount": { "currency_code": "<MONEDA>", "value":
"<DESCUENTO_APLICADO>" }
      }
    },
    "items": [{
      "name": "<NOMBRE_DEL_PRODUCTO>",
      "description": "<DESCRIPCION_DEL_PRODUCTO>",
      "sku": "<SKU_DEL_CATALOGO>",
      "quantity": "<CANTIDAD>",
      "unit_amount": { "currency_code": "<MONEDA>", "value":
"<PRECIO_UNITARIO_SIN_IMPUESTOS>" },
      "tax": { "currency_code": "<MONEDA>", "value":
"<IMPUESTO_POR_UNIDAD>" },
      "category": "PHYSICAL_GOODS"
    }],
    "shipping": {
      "name": { "full_name": "<NOMBRE_COMPLETO_DEL_DESTINATARIO>" },
      "address": {
        "address_line_1": "<CALLE_Y_NUMERO>", "address_line_2":
"<COLONIA_O_REFERENCIA>",
        "admin_area_2": "<CIUDAD_O_MUNICIPIO>", "admin_area_1":
"<ESTADO_CODIGO>",
        "postal_code": "<CODIGO_POSTAL>", "country_code":
"<PAIS_ISO_ALPHA2>"
      }
    }
  }
}]
}

```

💡 El payload de Create Order es idéntico para los cuatro métodos de pago (PayPal, PayLater, PayPal Credit, BCDC). El comercio no envía payment_source. PayPal asocia automáticamente la orden con el método de pago correspondiente al constructor de sesión que se invocó.

21.3 Reglas de validación del breakdown


```
amount.value === item_total + tax_total + shipping - discount
item_total    === Σ (item.unit_amount × item.quantity) por línea
tax_total     === Σ (item.tax × item.quantity) por línea
```

21.4 Capture Order

```
POST {PAYPAL_API_BASE}/v2/checkout/orders/<ORDER_ID>/capture
Authorization: Bearer <ACCESS_TOKEN>;
Content-Type: application/json
PayPal-Request-Id: <CAPTURE_REQUEST_ID>;
PayPal-Client-Metadata-Id: <CMID>;
{}
```

21.5 Contrato frontend → backend

 El frontend no construye el payload de Create Order. Envía únicamente referencias controladas: `_cmid`, `paymentMethod` y `cartId`. El backend reconstruye el payload desde el estado confiable del carrito autenticado. Aceptar montos enviados desde el cliente expone al comercio a manipulación de precios.

21.6 Backend — POST /api/orders

```
app.post("/api/orders", async (req, res) => {
  const { _cmid, paymentMethod, cartId } = req.body;

  // 1) Validación de entrada
  if (!isValidCMID(_cmid)) return res.status(400).json({ error:
"invalid cmid" });
  if (!isAllowedPaymentMethod(paymentMethod)) return res.status(400).json({ error:
"invalid paymentMethod" });
  const cart = await loadCartForUser(cartId, req.user); // valida ownership
  if (!cart) return res.status(404).json({ error: "cart
not found" });

  // 2) STC server-side, secuenciada, no bloqueante
  try {
    await callSTC({ cmid: _cmid, additionalData:
buildAdditionalDataFromTrustedState(req.user) });
  } catch (err) {
    logger.warn({ err, cmid: _cmid }, "STC failed; continuing checkout");
  }

  // 3) Construcción del payload desde estado confiable
  const orderPayload = buildOrderPayloadFromTrustedState({ cart, user: req.user });
  const accessToken = await getAccessToken();
  const createRequestId = await getOrCreateCreateRequestId(cartId);
```

```
// 4) Create Order con idempotencia por operación
const response = await fetch(`${process.env.PAYPAL_API_BASE}/v2/checkout/orders`, {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${accessToken}`,
    "Content-Type": "application/json",
    "PayPal-Request-Id": createRequestId,
    "PayPal-Client-Metadata-Id": _cmid
  },
  body: JSON.stringify(orderPayload)
});

const body = await response.json();
if (response.ok) await persistOrderContext({ orderId: body.id, cartId, cmid: _cmid,
createRequestId });
res.status(response.status).json(body);
});
```

Parte IV — Integración y operación

22. Orquestación End-to-End

22.1 Momento 1 — Inicialización del checkout (una sola vez por sesión)

```

1. cmid = generateCMID() [§12]
2. { clientId, sdkUrl } = GET /api/config [§10.4]
3. await loadSDK(sdkUrl) [§14.3]
4. sdkInstance = await paypal.createInstance({ clientId, components,
                                           pageType, locale }) [§15.1]
5. paymentMethods = await sdkInstance.findEligibleMethods({ [§15.2]
                                                           currencyCode })
6. Si isEligible('paylater'): paylaterDetails = getDetails('paylater') [§15.3]
7. Construir sesiones con sus callbacks: [§17-20]
   paypalSession / paylaterSession / creditSession / guestSession
8. Renderizar solo botones cuyo método es elegible. [§16.1]
9. Adjuntar addEventListener('click', ...) con patrón sin await. [§16.2]

```

22.2 Momento 2 — Clic del comprador (sin await antes de start)

```

btn.addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod }); // sin await
  session.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});

// createOrder(...) internamente llama a POST /api/orders, que ejecuta en orden:
// 1) Validar _cmid, paymentMethod y ownership de cartId.
// 2) await callSTC({ cmid, additionalData }) ← try/catch, no bloqueante
// 3) Construir orderPayload desde estado confiable
// 4) createRequestId = getOrCreateCreateRequestId(cartId)
// 5) POST /v2/checkout/orders + headers
// 6) Persistir { orderId, cartId, _cmid, createRequestId }
// 7) Responder al frontend con { id: orderId }

```

22.3 Momento 3 — onApprove y captura

```

onApprove(data) {
  const result = await captureOrder(data.orderId);
  showSuccess(result);
}

// POST /api/orders/<ORDER_ID>/capture + body._cmid = cmid
// → backend:
//   captureRequestId = getOrCreateCaptureRequestId(orderId)
//   headers: PayPal-Request-Id: <CAPTURE_REQUEST_ID> (distinto del de Create)
//           PayPal-Client-Metadata-Id: <CMID> (mismo de la sesión)

```

22.4 Regla mnemotécnica

Componente	Frecuencia
CMID	Una vez por sesión de checkout. Se reutiliza en STC, Create y Capture.
createInstance + findEligibleMethods	Una vez por sesión de checkout (después de cargar el SDK).
STC server-side	Antes de cada Create Order, dentro del handler POST /api/orders.
createRequestId	Un UUID por intención de Create Order (por cartId). Se reutiliza en reintentos del mismo Create.
captureRequestId	Un UUID por intención de Capture Order (por orderId). Se reutiliza en reintentos del mismo Capture. Distinto del createRequestId.
PayPal-Client-Metadata-Id	Mismo CMID en Create y Capture.
session.start(...)	Una vez por intento de cobro, dentro del handler del clic, sin await previo.

23. Puntos de Integración (Mapa de Endpoints)

Operación	Backend del comercio (sugerido)	API de PayPal
Configuración pública para el frontend	GET /api/config	— (no llega a PayPal)
Token OAuth (uso interno del backend)	(interno)	POST /v1/oauth2/token
Set Transaction Context (interno en POST /api/orders)	(interno)	PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}
Crear orden (incluye STC server-side)	POST /api/orders	POST /v2/checkout/orders
Consultar orden	GET /api/orders/:id	GET /v2/checkout/orders/{id}
Capturar orden	POST /api/orders/:id/capture	POST /v2/checkout/orders/{id}/capture

💡 No se expone un endpoint público /api/stc al frontend. STC se ejecuta server-side dentro de POST /api/orders para garantizar el orden estricto STC → Create Order y para que el frontend no tenga conocimiento del flujo de riesgo.

24. Consideraciones de Seguridad

24.1 Custodia de credenciales (NFR-02)

Asset	Almacenamiento	Transmisión
PAYPAL_CLIENT_SECRET	Variable de entorno del backend, gestionada por un secrets manager. Nunca en código fuente.	Solo backend → /v1/oauth2/token codificado en Authorization: Basic.
PAYPAL_MERCHANT_ID	Variable de entorno del backend.	Solo backend → URL de STC.
access_token	Memoria del proceso del backend. Caché con TTL inferior al expires_in. Nunca persistido.	Solo backend → API REST de PayPal en Authorization: Bearer.

24.2 Aislamiento PCI (NFR-01)

- En PayPal, PayLater y Credit, el flujo no involucra tarjeta del comprador desde la perspectiva del comercio.
- En BCDC, los datos de cuenta de tarjeta del comprador se capturan íntegramente en la UI hospedada por PayPal. Nunca entran al DOM ni al backend del comercio.
- El comercio puede certificarse bajo SAQ A.

24.3 CSP mínima recomendada (NFR-04)

```
script-src 'self' https://www.paypal.com https://www.paypalobjects.com
https://c.paypal.com;
frame-src https://www.paypal.com;
connect-src 'self' https://api-m.paypal.com https://api-m.sandbox.paypal.com;
img-src 'self' https://www.paypalobjects.com data:;
style-src 'self' https://www.paypalobjects.com 'unsafe-inline';
```

💡 En entornos Sandbox agregar adicionalmente <https://www.sandbox.paypal.com> a script-src y frame-src.

24.4 Logging y manejo de datos sensibles (NFR-08)

Permitido en logs:

- order_id, capture_id, invoice_id, custom_id, CMID, createRequestId, captureRequestId.

- status de la captura, código de error de PayPal y mensajes de validación.
- Nombre del método de pago elegido (paypal, paylater, credit, guest).

Prohibido en logs:

- access_token, client_secret, merchant_id, header Authorization completo.
- Bodies de respuesta de PayPal con secretos sin enmascarar.

25. Consideraciones Operativas

25.1 Métricas y observabilidad

Métrica	Granularidad	Alarma sugerida
Tasa de éxito de POST /v1/oauth2/token	Por minuto	< 99% en ventana de 5 min.
Latencia p95 de Create Order y Capture Order	Por minuto	Excede SLO interno.
Tasa de respuesta ≠ 200 en STC	Por minuto	> 5% en ventana de 5 min.
Tasa de éxito de captura por método (PayPal, PayLater, Credit, BCDC)	Por hora	Anomalía vs baseline histórico.
Tasa de onCancel por método	Por hora	Pico inusual sugiere fricción de UI.
Tasa de onError por método	Por hora	Indica problema técnico (SDK, red, configuración).
Tasa de UNPROCESSABLE_ENTITY en Create Order	Por minuto	> 0.5% indica bug en construcción del breakdown.

25.2 Idempotencia (NFR-06)

PayPal aplica idempotencia por operación: cada llamada a la API REST tiene su propia llave PayPal-Request-Id, y la misma llave se reutiliza únicamente ante reintentos del mismo request.

Operación	Llave recomendada	Reutilización
Create Order	createRequestId (UUID server-side, por cartId)	Reintentos del mismo Create (timeouts, errores de red, 5xx transitorios).
Capture Order	captureRequestId (UUID server-side, por orderId)	Reintentos del mismo Capture.
Nueva orden	Nuevo createRequestId	Aunque el CMID pueda conservarse en la misma sesión.
Nueva captura sobre nueva orden	Nuevo captureRequestId	—

```
async function getOrCreateCreateRequestId(cartId) {
  const existing = await store.get(`create-request-id:${cartId}`);
```



```

if (existing) return existing;
const id = crypto.randomUUID();
await store.set(`create-request-id:${cartId}`, id, { ttl: CHECKOUT_TTL });
return id;
}

async function getOrCreateCaptureRequestId(orderId) {
  const existing = await store.get(`capture-request-id:${orderId}`);
  if (existing) return existing;
  const id = crypto.randomUUID();
  await store.set(`capture-request-id:${orderId}`, id, { ttl: ORDER_TTL });
  return id;
}

```

 No usar el mismo UUID para Create y Capture. La separación por operación se alinea con la guía oficial de idempotencia de PayPal y previene comportamientos indefinidos.

25.3 Tolerancia a fallos

Servicio	Estrategia
POST /v1/oauth2/token	Reintentar una vez tras 401/5xx; si persiste, abortar y alertar.
POST /v2/checkout/orders	Reintentar con el mismo createRequestId ante 5xx o errores de red transitorios.
POST /v2/checkout/orders/{id}/capture	Igual que Create Order (mismo captureRequestId).
PUT /v1/risk/transaction-contexts/... (STC)	No reintentar. Loguear el fallo y continuar el checkout.
Carga del SDK v6	Si el <script> no carga, desactivar los botones de pago y mostrar mensaje de degradación al usuario.

25.4 Promoción Sandbox → Live

Punto	Cambio
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com → https://api-m.paypal.com
PAYPAL_SDK_URL	https://www.sandbox.paypal.com/web-sdk/v6/core → https://www.paypal.com/web-sdk/v6/core
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET	Credenciales de la app Live, no Sandbox.
PAYPAL_MERCHANT_ID	Merchant ID de la cuenta Live.

Punto	Cambio
application_context.return_url / cancel_url	URLs reales del dominio de producción del comercio.
CSP	Agregar/retirar dominios Sandbox según corresponda.

Parte V — Validación y gobierno

26. Estrategia de Pruebas y Entorno Sandbox

26.1 Casos de prueba mínimos (TC-01 a TC-16)

ID	Caso	Resultado esperado	Cumple RF/NFR
TC-01	Pago con cuenta PayPal (cuenta Sandbox válida)	Captura exitosa, <code>purchase_units[0].payments.captures[0].status === "COMPLETED"</code> .	RF-01, RF-04, RF-05
TC-02	Pago con PayLater en mercado elegible	Captura exitosa.	RF-03, RF-05
TC-03	Pago con PayPal Credit en mercado elegible	Captura exitosa.	RF-03, RF-05
TC-04	Pago con tarjeta vía BCDC (tarjeta Sandbox válida)	Captura exitosa. Inspección confirma que ningún payload del comercio contiene datos de cuenta de tarjeta.	RF-02, RF-04, RF-05, NFR-01
TC-05	Cancelación por el comprador (onCancel)	UI muestra mensaje sobrio, no se invoca captura.	RF-11
TC-06	Error técnico durante la sesión (onError)	UI muestra mensaje accionable, log con detalle del error.	RF-11
TC-07a	Retry técnico de Create por timeout/5xx (mismo cartId)	Mismo CMID, mismo createRequestId, mismo orderId devuelto (operación idempotente).	NFR-06
TC-07b	Nuevo intento del comprador tras rechazo o cancelación	Mismo CMID si sigue en la misma sesión, nuevo createRequestId, nueva orden.	RF-09
TC-08	Reintento idempotente de Capture (mismo captureRequestId)	PayPal devuelve la captura previa, no genera duplicado.	NFR-06
TC-09	STC responde 400	Checkout no se interrumpe; orden se crea correctamente.	RF-07, NFR-05
TC-10	Elegibilidad: mercado sin PayLater	Botón PayLater no se renderiza.	RF-12, NFR-12
TC-11	Elegibilidad: moneda sin PayPal Credit	Botón PayPal Credit no se renderiza.	RF-12, NFR-12

ID	Caso	Resultado esperado	Cumple RF/NFR
TC-12	Validación de transient activation	El pop-up/modal de PayPal abre sin bloqueo en Chrome, Firefox, Safari y Edge.	RF-06, NFR-14
TC-13	GET /api/config no expone secretos	Respuesta contiene clientId y sdkUrl; no contiene client_secret, merchant_id ni access_token.	NFR-02
TC-14	Validación CSP	Carga del SDK y apertura de la UI hospedada no producen violaciones de CSP.	NFR-04
TC-15	Cambio Sandbox → Live	Tras cambiar variables de entorno y recargar la página, el SDK opera contra el entorno correcto.	§25.4, §25.6
TC-16	Validación del CMID	Inspección confirma el mismo CMID en STC, PayPal-Client-Metadata-Id de Create Order y Capture Order.	RF-08

27. Convenciones de Nombres REST vs SDK

Concepto	API REST (snake_case)	SDK JavaScript v6 (camelCase)
Identificador de orden	id (en respuesta) / order_id	orderId (en data.orderId de onApprove)
Código de moneda	currency_code	currencyCode
Código de país	country_code	countryCode
Componente del SDK	(no aplica en REST)	components: ['paypal-payments', 'paypal-guest-payments']
Tipo de página	(no aplica en REST)	pageType: 'checkout'
Identificador de cliente PayPal	client_id (OAuth form-encoded)	clientId (en createInstance)
Modo de presentación de la UI	(no aplica en REST)	presentationMode: 'auto'
Header de correlación de riesgo	PayPal-Client-Metadata-Id (header HTTP)	(no aplica en SDK; se inyecta server-side)
Header de idempotencia	PayPal-Request-Id (header HTTP, llave por operación)	(no aplica en SDK; se inyecta server-side)

💡 Regla mental: Si el dato sale o entra de un endpoint REST (cuerpo JSON o headers HTTP), es snake_case o Pascal-Case con guiones (en headers). Si lo pasas a un método del objeto paypal.* o lo recibes en un callback del SDK, es camelCase.

28. Asunciones, Dependencias y Restricciones

28.1 Asunciones

ID	Asunción
A-01	El comercio dispone de un backend bajo su control donde puede custodiar PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID y emitir el access_token.
A-02	El comercio tiene un sistema de autenticación de usuarios y un servicio de carrito que produce un breakdown matemáticamente consistente.
A-03	El mercado destino soporta la moneda configurada y la elegibilidad de los métodos de pago deseados (PayPal, PayLater, Credit, BCDC).
A-04	El comprador usa un navegador moderno con soporte de pop-ups, transient activation, ES2017+ y CSP.
A-05	La cuenta de PayPal tiene activadas las habilitaciones comerciales listadas en §10.1.
A-06	El comercio gestiona el cambio de entorno Sandbox → Live mediante variables de entorno del backend; no requiere alternancia en runtime.

28.2 Dependencias externas

ID	Dependencia	Tipo
D-01	API REST de PayPal (api-m.paypal.com, api-m.sandbox.paypal.com).	Crítica — bloqueante.
D-02	PayPal Web SDK v6 (https://www.paypal.com/web-sdk/v6/core y su equivalente Sandbox).	Crítica — bloqueante.
D-03	Endpoint OAuth2 (/v1/oauth2/token).	Crítica — sin él no hay access_token.
D-04	Endpoint STC (/v1/risk/transaction-contexts).	Recomendada — no bloqueante.
D-05	Dominios www.paypal.com, www.paypalobjects.com, c.paypal.com (y equivalentes Sandbox) accesibles desde el navegador del comprador y permitidos en CSP.	Crítica — la UI hospedada vive en estos dominios.

28.3 Restricciones

ID	Restricción
R-01	El SDK v6 no admite parámetros de query string en su URL ni atributos data-* de configuración. Toda la configuración pasa por <code>createInstance(...)</code> .
R-02	El SDK v6 registra custom elements y no puede re-inicializarse en la misma página. Cambiar de entorno (Sandbox ↔ Live) requiere recargar.
R-03	El callback <code>createOrder</code> no puede ser <code>await</code> -eado dentro del handler del clic; el patrón obligatorio es construir la promesa y pasarla inmediatamente a <code>session.start(...)</code> .
R-04	El flujo canónico de BCDC se activa con <code>addEventListener('click', ...)</code> sobre <code><paypal-basic-card-button></code> , sin <code>targetElement</code> . La variante con <code>targetElement</code> (apertura programática / auto-start) requiere validación con el Integration Engineer.
R-05	El CMID es único por sesión de checkout y no debe reutilizarse entre compradores ni sesiones distintas.
R-06	PayPal-Request-Id se gestiona con llaves independientes por operación: <code>createRequestId</code> para Create Order, <code>captureRequestId</code> para Capture Order. Cada llave se reutiliza únicamente ante reintentos del mismo request.
R-07	El <code>client_secret</code> y el <code>merchant_id</code> no pueden, bajo ninguna circunstancia, alcanzar el navegador.
R-08	La integración cubierta por este SDD usa exclusivamente intent: "CAPTURE". AUTHORIZE queda fuera de alcance.

29. Riesgos y Mitigaciones

ID	Riesgo	Prob.	Impacto	Mitigación
RG-01	Exposición de PAYPAL_CLIENT_SECRET por commit accidental al repositorio.	Media	Crítico	Pre-commit hooks que detecten patrones de secretos; secrets manager; revisión de seguridad obligatoria. Plan de rotación inmediata si se detecta exposición.
RG-02	Exposición de access_token al frontend por bug en GET /api/config o en el envoltorio de respuestas de /api/*.	Baja	Crítico	Tests automatizados que validen que las respuestas no contienen access_token, client_secret ni merchant_id; revisión de código obligatoria.
RG-03	Bloqueo del pop-up por consumo accidental del transient activation (uso de await antes de session.start).	Alta	Alto	Code review con checklist explícita; lint rule custom; test E2E que confirme apertura del pop-up sin requerir interacción adicional del usuario.
RG-04	BCDC con marcado incorrecto (<button> HTML propio en lugar de <paypal-basic-card-button>, o componente 'paypal-guest-payments' ausente en createInstance).	Media	Alto	Test E2E que verifique que <paypal-basic-card-button> está presente, hidratado y dispara la sesión.
RG-05	Mezcla de credenciales Sandbox/Live (entorno cruzado).	Baja	Crítico	Validación del prefijo del CLIENT_ID y del PAYPAL_API_BASE al arrancar el backend; alarma si hay incoherencia.
RG-06	Order o Capture duplicada por reintento sin idempotencia, o llave reutilizada entre operaciones.	Media	Alto	Llaves separadas por operación (createRequestId, captureRequestId), persistidas server-side. Test que valide que las llaves de Create y Capture son distintas.
RG-07	Botones de métodos no elegibles renderizados al comprador.	Media	Medio	Filtrado obligatorio por findEligibleMethods antes de renderizar. Test E2E por mercado.
RG-08	Falla de CSP que bloquea la carga del SDK o la UI hospedada.	Media	Alto	CSP probada en staging por entorno; fallback de detección y mensaje claro al usuario si los scripts no cargan.

ID	Riesgo	Prob.	Impacto	Mitigación
RG-09	Bloqueo del checkout por error de STC (incumplimiento de NFR-05).	Baja	Crítico	STC implementado como no bloqueante con try/catch (§13.5). Test TC-09.
RG-10	Inicialización del SDK contra el entorno equivocado (URL o client_id cruzados).	Baja	Crítico	GET /api/config es la única fuente de clientId y sdkUrl para el frontend; el backend valida coherencia entre PAYPAL_API_BASE, PAYPAL_SDK_URL y credenciales.
RG-11	Re-inicialización del SDK en la misma página al cambiar entorno → conflicto de custom elements.	Baja	Alto	Forzar recarga de página al cambiar entorno; documentar la restricción para el equipo de operaciones.
RG-12	breakdown inconsistente → 422 UNPROCESSABLE_ENTITY.	Media	Medio	Función pura en el backend que arme breakdown e items con validación matemática previa al envío.
RG-13	Logs filtran secretos del backend (access_token, client_secret) sin enmascarar.	Media	Crítico	Wrapper de logging que aplique mascaramiento por defecto a claves sensibles; revisión de pipeline de observabilidad.
RG-14	El comercio cambia el clientId sin recargar la página → SDK queda con instancia obsoleta.	Baja	Medio	Cambios de clientId siempre disparan recarga (location.reload() o navegación).

30. Criterios de Aceptación y Checklist Pre-producción

30.1 Seguridad

- ☐ PAYPAL_CLIENT_SECRET y PAYPAL_MERCHANT_ID residen únicamente en variables de entorno del backend; nunca se incluyen en código fuente versionado ni se envían al navegador.
- ☐ Archivos de configuración con credenciales (.env, equivalentes) excluidos del control de versiones.
- ☐ El backend solo expone clientId y sdkUrl al frontend vía GET /api/config; nunca access_token, client_secret ni merchant_id.
- ☐ Logs no registran access_token, client_secret, headers Authorization completos ni campos sensibles sin enmascarar.
- ☐ Content Security Policy permite https://www.paypal.com, https://www.paypalobjects.com y https://c.paypal.com.
- ☐ TLS 1.2+ habilitado en producción.

30.2 Configuración

- ☐ PAYPAL_API_BASE apunta a https://api-m.paypal.com en Live.
- ☐ PAYPAL_SDK_URL apunta a https://www.paypal.com/web-sdk/v6/core en Live.
- ☐ PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET y PAYPAL_MERCHANT_ID corresponden al entorno Live.
- ☐ application_context.return_url y cancel_url son URLs reales y accesibles del dominio del comercio.
- ☐ El <script> del SDK no incluye atributos data-* ni parámetros de query string.
- ☐ El array components de createInstance contiene exactamente los componentes necesarios para los métodos expuestos.

30.3 Funcional

- ☐ findEligibleMethods se invoca en cada inicialización del checkout y solo se renderizan los botones cuyos métodos resultan elegibles.
- ☐ El handler del clic de cada botón no contiene await antes de session.start(...).
- ☐ BCDC: el HTML contiene <paypal-basic-card-container> con <paypal-basic-card-button> adentro; el listener de clic está sobre el <paypal-basic-card-button>.
- ☐ breakdown e items reflejan el estado real del carrito y cumplen las reglas de §21.3.
- ☐ PayPal-Request-Id: llaves independientes por operación (createRequestId y captureRequestId). Nunca se reutiliza la misma llave entre Create y Capture.
- ☐ Manejo de errores implementado en onError, onCancel y (en BCDC) onWarn.
- ☐ Los datos de payer, shipping y purchase_units provienen del estado del checkout, no de placeholders.

- ☐ Tras la captura, el backoffice registra order_id, capture_id, status, CMID, createRequestId, captureRequestId, invoice_id y custom_id.

30.4 Riesgo

- ☐ CMID generado una sola vez por sesión de checkout, propagado entre frontend y backend.
- ☐ STC se llama antes de cada Create Order (independientemente del método).
- ☐ additional_data de STC proviene de la sesión autenticada del comprador.
- ☐ STC no bloquea el checkout: errores se loguean y la transacción continúa.
- ☐ Header PayPal-Client-Metadata-Id presente en Create Order y Capture Order, con el mismo CMID.

30.5 Pruebas

- ☐ Matriz de casos de prueba TC-01 a TC-16 ejecutada y verde en Sandbox.
- ☐ Inspección DevTools confirma que ninguna llamada del navegador va directamente a api-m.paypal.com.
- ☐ Inspección DevTools confirma que GET /api/config no expone access_token, client_secret ni merchant_id.
- ☐ Pruebas manuales en Chrome, Firefox, Safari y Edge confirman que el pop-up/modal abre sin bloqueo en los cuatro métodos.

30.6 Operación

- ☐ Métricas de §25.1 instrumentadas y conectadas al sistema de observabilidad.
- ☐ Alarmas configuradas para los umbrales sugeridos.
- ☐ Runbook de incidentes documentado: falla de OAuth, falla de Create Order, anomalía en tasas de captura por método, caída del SDK.
- ☐ Plan de rotación de CLIENT_SECRET documentado y probado en Sandbox.
- ☐ Procedimiento documentado para promoción Sandbox → Live y para cambio coordinado de variables de entorno.

Apéndice A — Diagnóstico de Errores Frecuentes

Síntoma	Causa más probable	Cómo diagnosticar
401 Unauthorized en /v1/oauth2/token	CLIENT_ID / CLIENT_SECRET mal copiados o entorno cruzado (Sandbox vs Live).	Verificar que PAYPAL_API_BASE, PAYPAL_SDK_URL y las credenciales correspondan al mismo entorno.
El pop-up de PayPal queda bloqueado por el navegador	El handler del clic contiene await antes de session.start(...), consumiendo el transient activation.	Inspeccionar el handler: la promesa de Create Order debe construirse e inmediatamente pasarse a session.start(...) sin await previo.
BCDC: <paypal-basic-card-button> no se renderiza o queda inerte	El componente 'paypal-guest-payments' no está incluido en components de createInstance(...), o el HTML se montó antes de que el SDK terminara de cargar.	Confirmar que 'paypal-guest-payments' está en components. Asegurarse de invocar createInstance(...) antes de mostrar el contenedor.
BCDC: el clic no dispara la sesión	El listener se adjuntó a un elemento incorrecto (ej. al <paypal-basic-card-container> en vez de al <paypal-basic-card-button>), o el elemento se reemplazó por el SDK después de adjuntar el listener.	Adjuntar el listener al <paypal-basic-card-button> después de que el SDK haya terminado de hidratar los custom elements.
findEligibleMethods devuelve un set vacío	Combinación client_id + currencyCode + país del comprador sin habilitaciones suficientes.	Verificar la habilitación comercial en el Developer Dashboard; verificar currencyCode ISO 4217; consultar al Integration Engineer.
Botón PayLater renderizado pero la sesión falla al activarse	Se renderizó sin validar paymentMethods.isEligible('paylater').	Filtrar por isEligible(...) antes de mostrar cada botón.
422 UNPROCESSABLE_ENTITY en Create Order	breakdown que no cuadra con amount.value y/o suma de items.	Aplicar reglas de §21.3 manualmente; PayPal devuelve el campo específico inconsistente en la respuesta.
Captura ejecuta dos veces ante un retry del navegador	El backend genera un captureRequestId nuevo por cada llamada en lugar de buscarlo en su store por orderId.	Implementar getOrCreateCaptureRequestId(orderId) con persistencia para devolver el mismo UUID en reintentos del mismo Capture.
Mensajes contradictorios al reintentar Create tras un timeout	El backend genera un createRequestId nuevo en el reintento → PayPal crea una orden adicional.	Persistir createRequestId por cartId y reutilizarlo en reintentos del mismo Create.

Síntoma	Causa más probable	Cómo diagnosticar
Cambio de entorno (Sandbox ↔ Live) deja la página rota	Se intentó re-inicializar el SDK sin recargar la página.	Forzar recarga (location.reload()) tras cambiar variables de entorno.
GET /api/config devuelve access_token o client_secret	Bug en la implementación del endpoint: se serializa el objeto completo de configuración.	Implementar lista blanca de campos ({ clientId, sdkUrl }) en lugar de res.json(config) directo.
STC responde 400	Tipo de campo incorrecto en additional_data.	Validar formatos según §13.3 (especialmente sender_create_date y sender_phone).
STC responde 5xx pero el checkout se interrumpe	El handler POST /api/orders está propagando el error de STC en lugar de absorberlo dentro de su try/catch.	El bloque await callSTC(...) debe estar envuelto en try/catch; cualquier excepción se loguea como warn y se continúa con Create Order.
paypal.createInstance is not a function	El <script> del SDK no terminó de cargar antes de la llamada.	Esperar el onload del <script> o usar el helper loadSDK(...) con Promise.
onApprove recibe data.orderId undefined	Forma de la promesa devuelta por createOrder incorrecta.	El callback debe devolver { orderId: <ORDER_ID> } (no { id: ... } ni el order_id desnudo).
El SDK no muestra la UI en el idioma esperado	locale mal formado o no soportado en el mercado del comprador.	Usar formato BCP-47 con guion ('es-MX', no 'es_MX').

Apéndice B — Limitaciones y Trabajo Futuro

B.1 Limitaciones conocidas

- Cambio de entorno en runtime no es soportado por el SDK v6 (registra custom elements). La selección de Sandbox/Live debe hacerse en arranque del backend.
- AUTHORIZE + captura diferida queda fuera de alcance. Esta integración usa exclusivamente intent: "CAPTURE".
- Capacidades de tarjeta avanzadas (captura en iframes propios, control explícito de autenticación reforzada y su resultado, pagos a meses, reuso de tarjeta entre sesiones) no son parte de esta integración.
- La variante con targetElement para activación programática de BCDC (auto-start) no es el flujo canónico de este SDD y requiere validación con el Integration Engineer.

B.2 Trabajo futuro sugerido

Iniciativa	Descripción
Operaciones de back-office	SDD complementario para webhooks (PAYMENT.CAPTURE.COMPLETED, PAYMENT.CAPTURE.DENIED, CUSTOMER.DISPUTE.CREATED), reembolsos parciales/totales, settlement y conciliación financiera.
Métodos de pago adicionales	Extensión de la solución para incluir billeteras locales u otros métodos disponibles en el SDK v6, agregando los componentes correspondientes a createInstance.
Captura diferida (AUTHORIZE)	Para modelos de negocio (preorders, dropshipping) donde la captura se realiza tras confirmación de inventario o envío.
Industry pack	Adopción del set extendido de additional_data de STC correspondiente a la vertical del comercio (Travel, Gaming, Financial Services, etc.).

Solution Design Document para la integración de PayPal Checkout (PayPal, PayLater, PayPal Credit) y Branded Card-Direct Checkout (BCDC) sobre el PayPal JavaScript SDK v6 con arquitectura server-side.